Gauthier Vermandel ○ *gauthier.vermandel@dauphine.psl.eu*

A Note on

DB·nomics with MATLAB

Content of the Handout

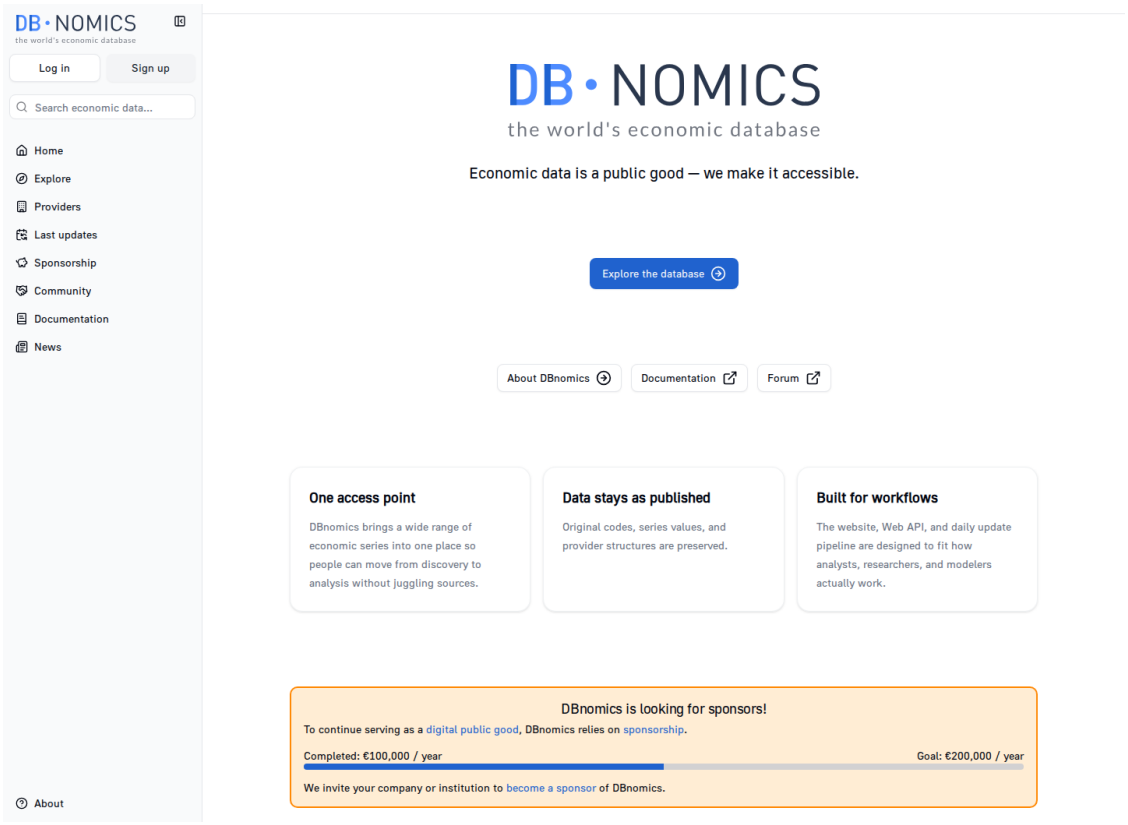
1 What is DB·nomics? ..... p. 1
2 Installation and first request ..... p. 4
3 One series ..... p. 4
4 Several countries ..... p. 5
5 Several series and alignment ..... p. 5
6 Plotting ..... p. 6

1. What is DB·nomics?

DB·nomics gives access to datasets supplied by many institutions through one web interface. It does not replace the original provider: before using a series, verify its definition, unit, frequency, seasonal adjustment, coverage, revision policy and licence. Record the source provider and the access date in every reproducible empirical project.

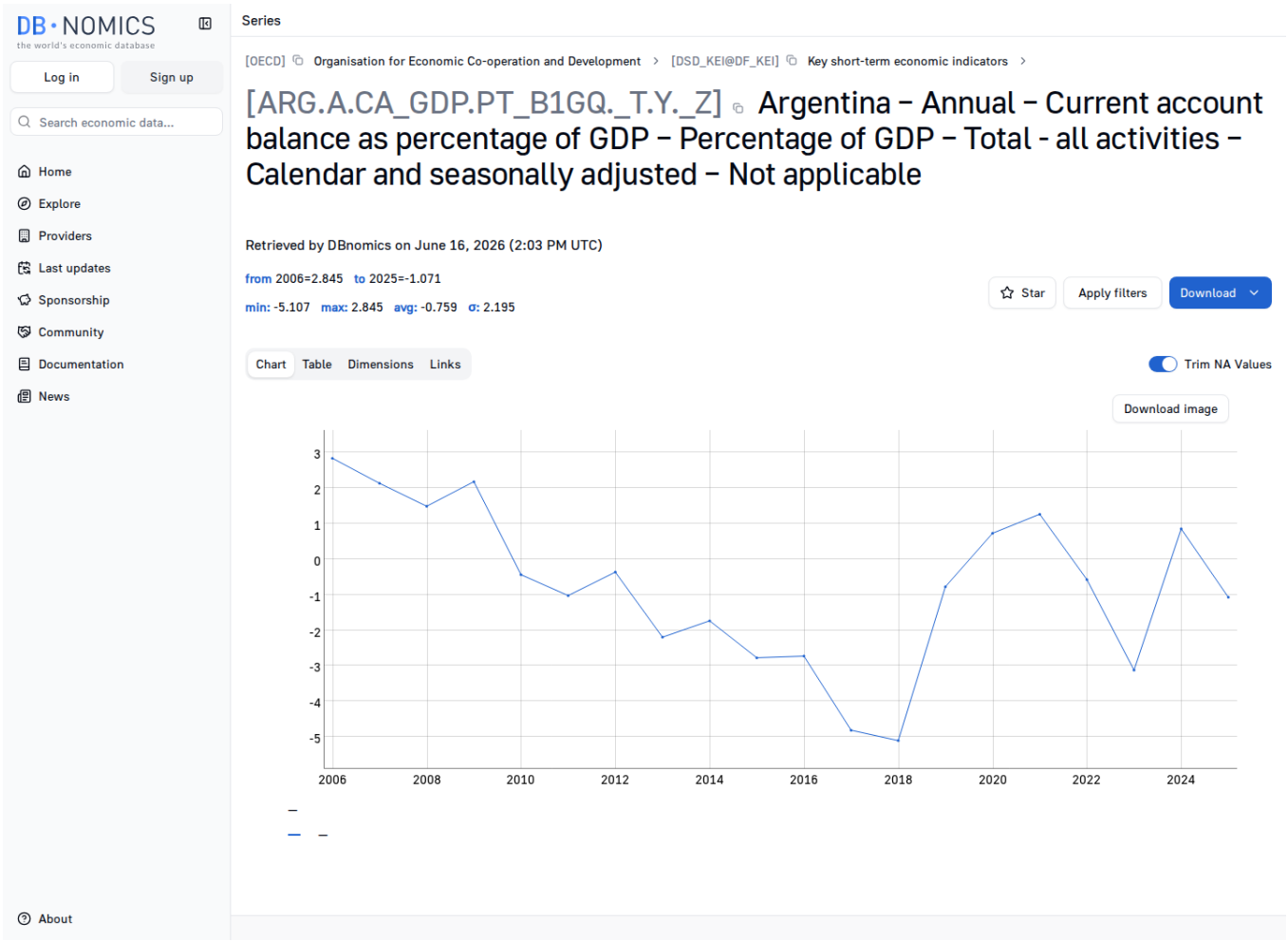
Find a series in the current interface

Open `https://db.nomics.world/`. Use the left-hand *Search economic data* box, *Explore the database*, or *Providers*. Search by concept, country and provider, then open a dataset and a series; a search result is not yet a data definition. No login is needed for the public data used here.



DBnomics home page, captured on 11 November 2026.

On the series page, the top breadcrumb identifies the provider and dataset. The bracketed code before the title identifies the series within that dataset. Read the full title, then use *Chart*, *Table*, *Dimensions* and *Links* to inspect the observations, definitions and original source. *Download* exports data; *Apply filters* opens a separate editor and is not needed to retrieve the original observations in MATLAB.



OECD example, captured on 11 November 2026 (browser address bar not shown).

Page URL (shown explicitly, not part of the screenshot):
https://db.nomics.world/OECD/DSD_KEI@DF_KEI/ARG.A.CA_GDP.PT_B1GQ._T.Y._Z
Public series token to copy into MATLAB:

```
'OECD/DSD_KEI@DF_KEI/ARG.A.CA_GDP.PT_B1GQ._T.Y._Z'
```

The token (also called a *slug* or series identifier) is the complete `provider/dataset/series` path after the website domain. Keep the `@`, underscores, periods and capitalisation exactly as shown; omit the domain and any query string. Here the provider is `OECD`, the dataset is `DSD_KEI@DF_KEI`, and the final component is `ARG.A.CA_GDP.PT_B1GQ._T.Y._Z`. Copying only that final component is not enough for `call_dbnomics`.

This public token is not a secret API key or authentication token. The example is Argentina's annual current-account balance as a percentage of GDP, not GDP growth. On 11 November 2026 the page and API both returned annual observations from 2006 to 2025. Check these metadata again when reusing it. For reproducibility, record the token, page URL, provider, access date, frequency, units, transformations and missing-value treatment alongside code.

2. Installation and first request

The only MATLAB interface used in this note is `call_dbnomics.m`. Download the repository linked below. The function is in its `code/` folder; add that folder to the MATLAB path. In the standalone guide source folder, the same function is at the root. MATLAB must have `webread`, `datetime`, `string` and Internet access. The examples are tested with MATLAB R2025a; no additional toolbox is required.
https://github.com/Vermandel/dbnomics-teaching No DBnomics account, API key or credential is required.
Direct download: `call_dbnomics.m` (required function). Save it in a local `code/` folder and the examples linked below in a sibling `examples/` folder. Keep the original filenames. These links open raw files directly; if code is displayed, use the browser's *Save as*. No account is needed.

```
addpath('code') % from the downloaded repository root
[output_mat, output_table, dates_nb] = call_dbnomics( ...
    'Eurostat/prc_hicp_manr/M.RCH_A.TOT_X_NRG.AT');
```

The function signature is

```
[output_mat, output_table, dates_nb] = call_dbnomics(slug1, slug2, ...)
```

If `RequestFailed` occurs, check the connection and retry the page in a browser: a temporary service failure does not establish that the token is wrong. If `InvalidSlug`, `MissingSeries` or `NoSeries` occurs, check the full three-part token.

All requested slugs must have the same frequency. The function downloads the series, sorts their periods, aligns them on the union of dates and leaves a missing observation as `NaN`.

- `output_mat`: first column = MATLAB `datenum` dates; remaining columns = observations.
- `output_table`: same observations in a MATLAB table; row names are period labels and column names are the final dimension codes (for example `AT` and `BE`). MATLAB makes repeated names unique.
- `dates_nb`: a readable numeric date vector: 2000, 2000.25, 2000.50 for quarterly data; $y+(m-1)/12$ for monthly data. Annual data use the year; daily data use MATLAB serial dates (not fractional years).

3. One series

Download: `example_single_series.m`. Austrian inflation.

The first example downloads Austrian inflation. The second matrix column holds the observations; its first column holds the dates.

```
[mat, tab, dates] = call_dbnomics( ...
    'Eurostat/prc_hicp_manr/M.RCH_A.TOT_X_NRG.AT');
inflation_AT = mat(:,2);
```

Inspect `tab(1:5,:)` and the last rows before transforming a series. Do not confuse a rate of change, an index level and a seasonally adjusted level.

4. Several countries

Download: example_multiple_countries.m. Several countries in one request.

Within a dimension, `+` expands several codes. The following request, from the historical examples, returns four columns: `AT`, `BE`, `DE` and `FR`.

```
[mat, tab] = call_dbnomics( ...
    'Eurostat/prc_hicp_manr/M.RCH_A.TOT_X_NRG.AT+BE+DE+FR');
```

If country coverage differs, the unmatched dates are represented by `NaN`. A missing value is not zero: do not interpolate, forward-fill or drop dates without a documented economic and statistical reason.

5. Several series and alignment

Download: example_multiple_series.m. Aligned series and missing dates.

Pass several slugs as separate MATLAB arguments. This example requests two monthly HICP measures for Austria and Germany:

```
[mat, tab] = call_dbnomics( ...
    'Eurostat/prc_hicp_manr/M.RCH_A.TOT_X_NRG.AT+DE', ...
    'Eurostat/prc_hicp_manr/M.RCH_A.TOT_X_TBC.AT+DE');
complete = all(~isnan(mat(:,2:end)),2);
```

The function does not silently aggregate or disaggregate. Download different frequencies separately, then state and justify any conversion. Daily, monthly, quarterly and annual period conventions are supported; `UnsupportedFrequency` requires inspecting the actual period format. When names are repeated, use `tab.Properties.VariableNames` to inspect the resulting column names rather than assuming an order.

6. Plotting

Both scripts download exactly two monthly series: Austrian and Belgian HICP annual inflation excluding energy. Their units and frequency are comparable. The function aligns them on the sorted union of observation dates; matrix columns 2 and 3 retain the order of the two arguments. Table values use that same alignment. A missing value remains `NaN` and creates a gap in the line: never replace it with zero merely to obtain a continuous plot.

A datetime time vector

Download and run `example_plot_datetime.m` (two series, datetime axis). Convert the first matrix column from MATLAB serial dates to `datetime`. Do not parse the table row names with an assumed date format: their labels depend on frequency.

```
[mat, tab, dates_nb] = call_dbnomics( ...
    'Eurostat/prc_hicp_manr/M.RCH_A.TOT_X_NRG.AT', ...
    'Eurostat/prc_hicp_manr/M.RCH_A.TOT_X_NRG.BE');
assert(size(mat,2) == 3 && height(tab) == size(mat,1));
assert(isequaln(tab{:,:}, mat(:,2:3)));
t = datetime(mat(:,1), 'ConvertFrom', 'datenum');
assert(numel(t) == size(mat,1) && all(diff(mat(:,1)) > 0));
figure
plot(t, mat(:,2), '-', t, mat(:,3), '--', 'LineWidth', 1.2)
grid on
legend('Austria', 'Belgium', 'Location', 'best')
title('HICP inflation excluding energy')
ylabel('Annual percentage change')
xlabel('Date')
```

A numeric time vector

Download and run example_plot_numeric.m (two series, numeric axis). Here the third output is a numeric vector $y + (m-1)/12$: January is y, February is $y + 1/12$. This is a readable monthly time scale, not a MATLAB `datenum`; do not pass it to `datetick` or convert it with `ConvertFrom='datenum'`.

```
[mat, tab, dates_nb] = call_dbnomics( ...
    'Eurostat/prc_hicp_manr/M.RCH_A.TOT_X_NRG.AT', ...
    'Eurostat/prc_hicp_manr/M.RCH_A.TOT_X_NRG.BE');
assert(size(mat,2) == 3 && height(tab) == size(mat,1));
assert(isequaln(tab{:,:}, mat(:,2:3)));
t = dates_nb; % year + (month - 1)/12, not MATLAB serial dates
assert(numel(t) == size(mat,1) && all(diff(mat(:,1)) > 0));
figure
plot(t, mat(:,2), '-', t, mat(:,3), '--', 'LineWidth', 1.2)
grid on
legend('Austria', 'Belgium', 'Location', 'best')
title('HICP inflation excluding energy')
ylabel('Annual percentage change')
xlabel('Year (fractional numeric scale)')
```

Both examples use the returned dates, never a hand-made vector whose length may differ from the downloaded sample. If a complete-case sample is needed, apply the *same* logical mask to time and to both value columns:

```
keep = all(~isnan(mat(:,2:3)),2);
t_complete = t(keep);
y_complete = mat(keep,2:3);
```

This removes incomplete rows; it is a substantive sample choice, not an alignment repair. Inspect first and last dates, observation counts and units before analysis, and cite the original provider as well as DBnomics.

Legacy entry point: example_plot_inflation.m. This compatibility script runs `example_plot_datetime.m`; download both into the same `examples/` folder.